Microfreelance

Tested marketplace workflows and an interactive product preview

Product and engineering review

Evidence version 435dafaa3164

1 Purpose and implementation boundary

Microfreelance combines a working PHP marketplace with a separate interactive preview of prompt-driven work. This paper documents observable behavior and the evidence supporting it. It does not claim that simulated execution is a production agent service.

Implemented marketplace. Registration and sessions, deposits with administrator credit, funded orders, responses, proposals, work submission, customer acceptance and once-only payouts operate against JSON persistence. Tests use temporary synthetic data; repository data is neither copied into the browser fixture nor changed by the publication build.

UI-only preview. All additional agents, API gates, workspaces, credits and payments are browser-only simulations. No provider, Tether or payment integration executes. A reviewer can explore a full journey, including failure and revision states. The two workspace fixtures demonstrate separate UI state, not server-side tenant authorization. Mock API keys never grant access to a real endpoint.

Review map

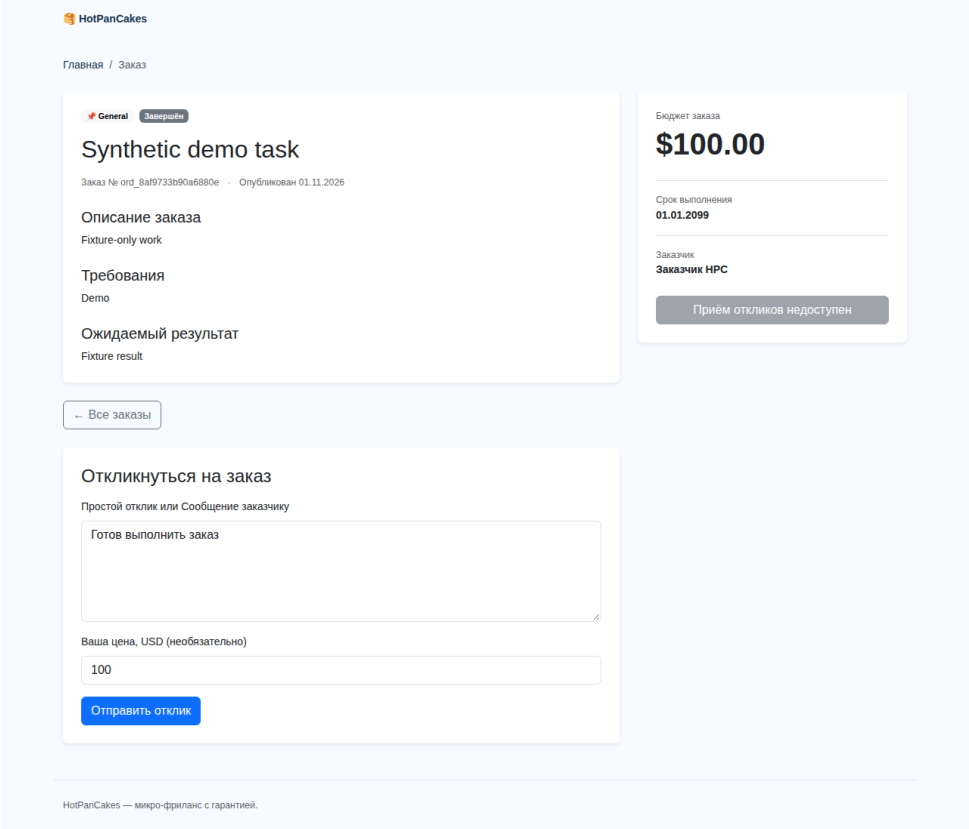
Read the implemented workflow first, then the UI-only journey. The workflow guide links every published journey to its test, recording and screenshots. Publication counts are derived from the same passing browser run as the images.

Area	Evidence and interpretation
Marketplace	Persisted outcomes, permission failures and replay protection.
Preview	Local UI transitions, reset behavior and no marketplace writes.
Publications	Same-commit inputs, screenshot checksums and PDF checks.

2 The existing financial lifecycle

A customer registers, creates a deposit and reports payment. An administrator confirms credit. The customer creates an order; the application reserves funds. A freelancer responds, the customer sends a proposal, and the selected freelancer accepts it. Submission moves work into review. Customer acceptance releases the reservation and settles the payout once.

A USD 100 contract reserves USD 104. Acceptance pays USD 96 to the contractor and USD 8 in combined fees. Tests also reject unauthorized assignment and submission, prevent duplicate credit and payout, and preserve financial data when malformed JSON is encountered. The proposal expiry currently implemented is 30 minutes.

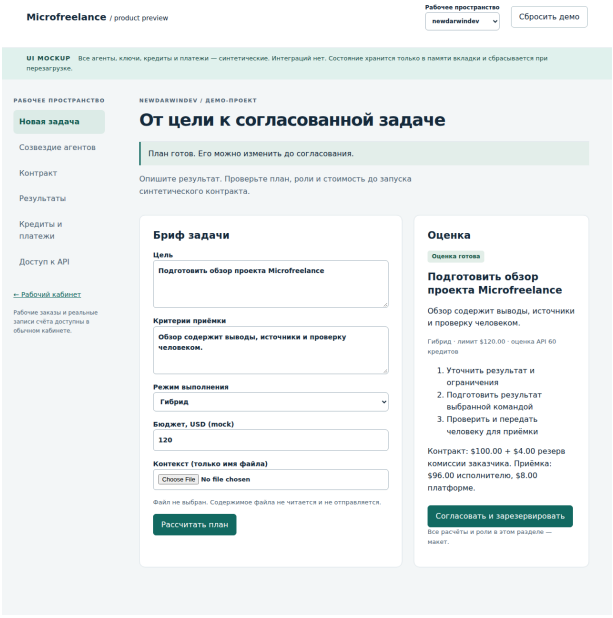


Completed synthetic marketplace order. Synthetic browser evidence, commit `435dafaa3164`.

3 The preview journey

The authenticated dashboard links to Product preview. A reviewer writes a goal and acceptance criteria, chooses human, AI or hybrid execution, and sets a USD budget cap. An optional context file contributes its name only; the browser does not read or upload its contents. The plan remains editable until approved.

The fixed example uses a USD 100 contract and requires a cap of at least USD 104. Estimated API usage is zero credits for human work, 40 for AI, or 60 for hybrid work. Synthetic available balances begin at USD 200 and 1,200 API credits per workspace. Credits and USD are independent units.

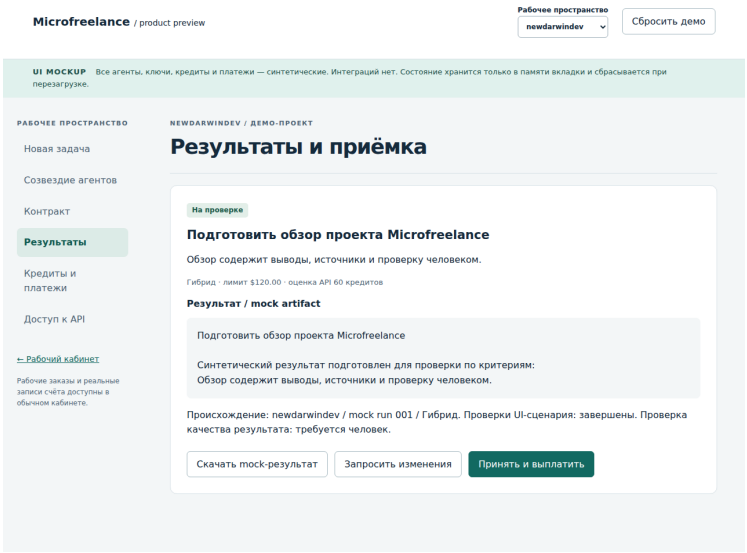


Editable brief and estimated plan. Synthetic browser evidence, commit 435dafaa3164.

Approval reserves the contract funds and estimated credits. A coordinator, builder, reviewer, contractor and customer are displayed as available roles. Session identifiers are deterministic mock labels. Execution advances through explicit reviewer actions; no background process runs.

A failed execution can be retried without a second reservation. Completion consumes 60 percent of the estimated credits and releases unused credits. The result is a downloadable synthetic text artifact. Run completion, human acceptance and settlement remain separate states. Revision reopens review without extra simulated usage charges. These fixed demo rules are not proposed production pricing.

4 Review, cancellation and settlement



Synthetic result awaiting human acceptance. Synthetic browser evidence, commit `435dafaa3164`.

Customer acceptance records the mock payout once. With hybrid execution, 60 credits were reserved, 36 consumed and 24 released; the remaining credit balance is 1,164. USD available is 96 after reservation and remains 96 after settlement. The payout and fee displays show USD 96 and USD 8 respectively.

Cancellation releases unused USD and API reservations. Already consumed API credits are not returned. Reloading the page or selecting Reset demo restores fixtures; nothing is persisted. Switching workspaces retains separate in-memory scenarios within the same tab.

5 Access states and limitations

The API access screen simulates allowed access, no session, insufficient permissions, revoked keys, missing entitlement, exhausted quota and insufficient credits. Each denied state blocks approval in the preview. Key rotation restores the allowed UI state; revocation selects the revoked state. These controls do not replace authentication or implement an API security boundary.

The preview makes no network request after its page and local assets load. Regression tests compare marketplace JSON fixtures before and after the journey. Actual login and the original marketplace retain their existing server behavior. No new schema migration is required.

6 Architecture and evidence

The existing application remains PHP with JSON files and the shared financial lock. The preview is an authenticated PHP shell plus local CSS and JavaScript. Each workspace has an in-memory state machine. Its contract moves through draft, estimate, funded, running, failed, review, revision, accepted or cancelled states.

The browser run supporting these publications passed 8 functional tests, 11 preview tests and 3 recorded demonstrations. Functional tests validate existing behavior; preview tests validate simulated UI interactions. Neither is represented as full application code coverage. PHPUnit coverage remains a separate report with uncovered legacy PHP included.

The workflow catalog maps each published journey to a passing test and recording. Screenshots have SHA-256 checksums. Publication preparation checks the browser report, screenshot checksums and commit identity before compiling. Both PDFs share terminology and version macros, and are compiled with LuaLaTeX through Docker and Make. The full evidence manifest, toolchain package list and editable TeX source bundle accompany the PDFs.

7 Future integration boundaries

A later implementation would need explicit server-side tenant membership, scoped authorization, credential storage, idempotent execution and usage events, provider pricing, payment reconciliation and verified Tether adapters. Those capabilities remain deferred. The current preview provides a reviewable interaction model; it is not a basis for charging money or running real agents.

Known marketplace gaps remain documented in the workflow guide: linked missing routes, placeholder pages, financial concurrency and injected application write failures. Passing tests demonstrate the scenarios exercised; they do not establish comprehensive correctness or production readiness.

8 Repository references

- `docs/WORKFLOWS.md`: steps, expected outcomes, limitations and test references.
- `docs/BUILD.md`: containerized publication commands and toolchain updates.
- `evidence-manifest.json`: commit, workflow references, screenshots and browser counts.
- Private GitHub Actions artifacts: recordings, test reports, coverage and the HTML dashboard.

Synthetic browser evidence, commit `435dafaa3164`.